

## Pemrograman Dasar TLS

### A. Problem 1 : Seleksi Astronot Terakhir

#### 1. Pseudocode

MULAI

BACA N, K           // N = jumlah astronot, K = nilai awal eliminasi

BUAT larik aktif[0..N-1], semua diisi true

sisa = N

idx = 0           // posisi astronot nomor 1 (titik awal hitungan)

SELAMA sisa > 1:

    hitungan = 1

    SELAMA hitungan < K:

        idx = (idx + 1) MOD N

        JIKA aktif[idx] = true MAKA:

            hitungan = hitungan + 1

    nomorDieliminasi = idx + 1

    CETAK nomorDieliminasi

    aktif[idx] = false

    sisa = sisa - 1

    JIKA nomorDieliminasi MOD 2 == 0 MAKA:

        K = K + 2

    SELAIN ITU:

        K = K - 1

    JIKA K < 2 MAKA:

        K = 2

    // pindah ke astronot aktif berikutnya sebagai titik awal hitungan baru

    next = (idx + 1) MOD N

    SELAMA aktif[next] = false:

        next = (next + 1) MOD N

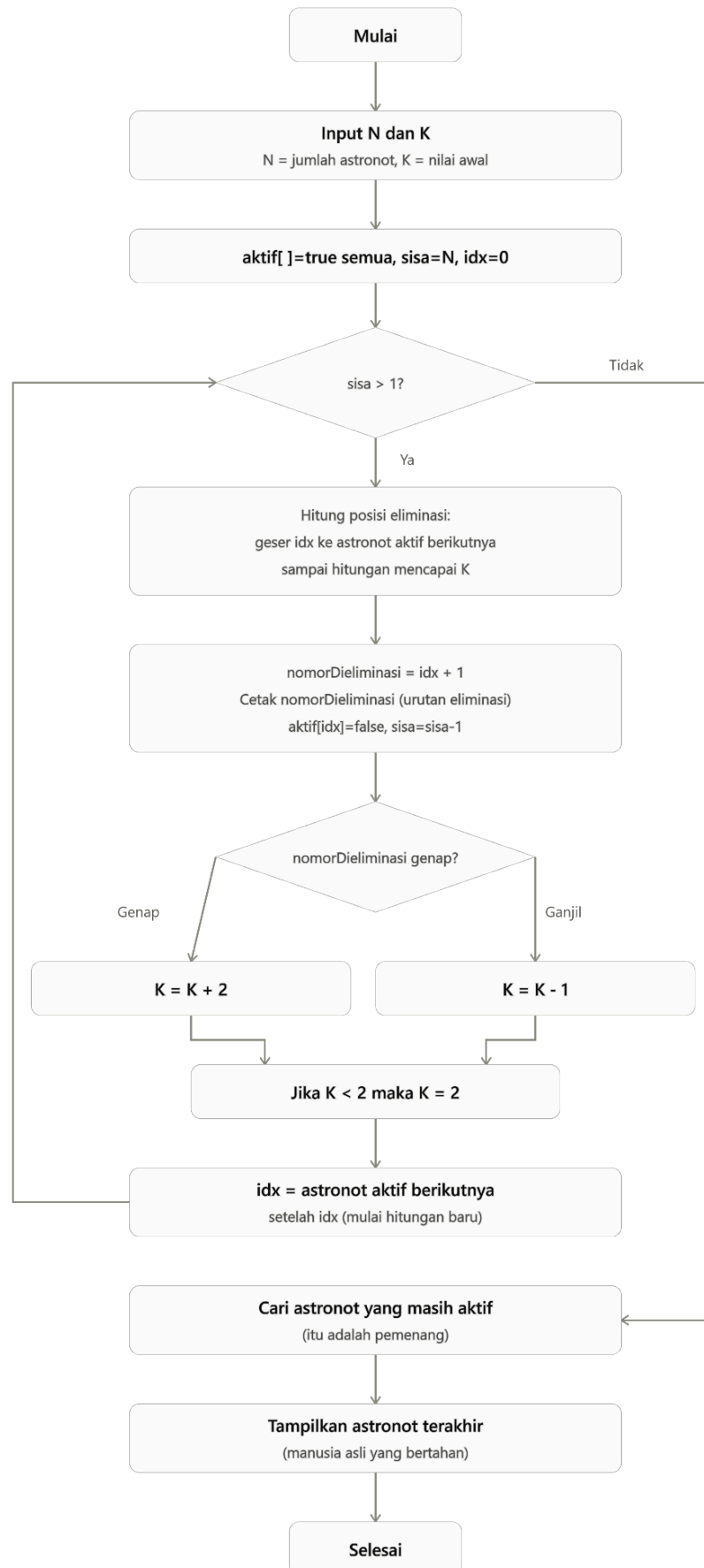
    idx = next

CARI astronot yang masih aktif -> itu pemenang (manusia asli)

CETAK astronot terakhir

SELESAI

## 2. Flowchart



### 3. Penjelasan

#### **Problem 1: The Last Astronaut (Josephus dengan K dinamis)**

BACA  $N$ ,  $K$

**Langkah 1** — Program membaca dua input:  $N$  (jumlah astronot yang berdiri melingkar, diberi nomor 1 sampai  $N$ ) dan  $K$  (nilai awal sistem eliminasi).

BUAT larik aktif[0.. $N$ -1], semua diisi true

sisa =  $N$

idx = 0

**Langkah 2** — Disiapkan larik **aktif** untuk menandai astronot mana saja yang masih ada di lingkaran (semua mulai dari **true**/aktif). Variabel **sisa** melacak berapa banyak astronot yang tersisa, dan **idx** menunjuk ke astronot nomor 1 sebagai titik awal hitungan pertama.

SELAMA sisa > 1:

**Langkah 3** — Ini loop utama. Proses eliminasi terus berulang selama masih ada lebih dari satu astronot di lingkaran.

hitung = 1

SELAMA hitung <  $K$ :

idx = (idx + 1) MOD  $N$

JIKA aktif[idx] = true MAKA:

hitung = hitung + 1

**Langkah 4** — Ini bagian "menghitung maju". **idx** sudah dianggap hitungan ke-1 (posisi awal). Loop dalam ini menggeser **idx** satu per satu di sekeliling lingkaran (pakai modulo  $N$  supaya kembali ke awal saat melewati astronot terakhir), tapi **hanya astronot yang masih aktif yang dihitung**. Proses berhenti begitu hitungan mencapai  $K$  — pada titik itu **idx** menunjuk ke astronot yang harus dieliminasi.

nomorDieliminasi = idx + 1

CETAK nomorDieliminasi

aktif[idx] = false

sisa = sisa - 1

**Langkah 5** — Nomor astronot yang kena hitungan ke- $K$  dicatat (**idx + 1** karena index array mulai dari 0, sedangkan nomor astronot mulai dari 1), lalu dicetak sebagai

bagian dari urutan eliminasi. Statusnya diubah jadi tidak aktif, dan jumlah sisa dikurangi 1.

JIKA nomorDieliminasi MOD 2 == 0 MAKA:

$K = K + 2$

SELAIN ITU:

$K = K - 1$

JIKA  $K < 2$  MAKA:

$K = 2$

**Langkah 6** — Ini aturan perubahan K sesuai soal: kalau nomor yang baru dieliminasi **genap**, nilai K ditambah 2; kalau **ganjil**, K dikurangi 1. Setelah itu dicek batas bawahnya — kalau hasilnya kurang dari 2, K dipaksa jadi 2.

$\text{next} = (\text{idx} + 1) \text{ MOD } N$

SELAMA aktif[next] = false:

$\text{next} = (\text{next} + 1) \text{ MOD } N$

$\text{idx} = \text{next}$

**Langkah 7** — Titik awal untuk putaran hitungan berikutnya dicari: mulai dari posisi tepat setelah astronot yang baru dieliminasi, lalu digeser terus sampai menemukan astronot yang masih aktif. Posisi ini disimpan ke **idx**, lalu kembali ke Langkah 3 (loop **SELAMA sisa > 1**).

CARI astronot yang masih aktif -> itu pemenang

CETAK astronot terakhir

SELESAI

**Langkah 8** — Setelah loop berhenti (tersisa satu astronot), program mencari astronot yang statusnya masih **aktif = true** di dalam larik — itulah "manusia asli" yang bertahan, lalu ditampilkan sebagai hasil akhir.

## B. Problem 2 : Memecahkan Sandi Rahasia Alien

### 1. Pseudocode

MULAI

BACA teks (larik karakter, diakhiri karakter null)

HITUNG panjang teks secara manual:

$\text{panjang} = 0$

SELAMA teks[panjang] != '\0':

$\text{panjang} = \text{panjang} + 1$

hurufSebelumnya = kosong

sudahAdaHurufPertama = SALAH

UNTUK  $i = 0$  SAMPAI  $\text{panjang} - 1$ :

$c = \text{teks}[i]$

// Cek huruf besar/kecil tanpa fungsi isalpha()

$\text{hurufBesar} = (c \geq 'A' \text{ DAN } c \leq 'Z')$

$\text{hurufKecil} = (c \geq 'a' \text{ DAN } c \leq 'z')$

JIKA (BUKAN  $\text{hurufBesar}$ ) DAN (BUKAN  $\text{hurufKecil}$ ) MAKA:

$\text{hasil}[i] = c$  // bukan huruf, dibiarkan apa adanya

LANJUT ke  $i$  berikutnya

JIKA  $\text{sudahAdaHurufPertama} = \text{SALAH}$  MAKA:

$\text{hasil}[i] = c$  // huruf pertama tidak berubah

$\text{hurufSebelumnya} = c$

$\text{sudahAdaHurufPertama} = \text{BENAR}$

SELAIN ITU:

// Tentukan nilai posisi alfabet ( $A/a=1 \dots Z/z=26$ ) secara manual

JIKA  $\text{hurufSebelumnya}$  huruf besar MAKA:

$\text{nilaiSebelum} = \text{hurufSebelumnya} - 'A' + 1$

SELAIN ITU:

$\text{nilaiSebelum} = \text{hurufSebelumnya} - 'a' + 1$

JIKA  $\text{hurufBesar}$  MAKA:

$\text{nilaiSekarang} = c - 'A' + 1$

SELAIN ITU:

$\text{nilaiSekarang} = c - 'a' + 1$

// Rumus pergeseran dengan wrap-around modulo 26

$\text{nilaiBaru} = ((\text{nilaiSekarang} + \text{nilaiSebelum} - 1) \text{ MOD } 26) + 1$

// Ubah nilai posisi kembali menjadi huruf, sesuai case aslinya

JIKA  $\text{hurufBesar}$  MAKA:

$\text{basis} = 'A'$

SELAIN ITU:

$\text{basis} = 'a'$

$\text{terenripsi} = (\text{nilaiBaru} - 1) + \text{basis}$

$\text{hasil}[i] = \text{terenripsi}$

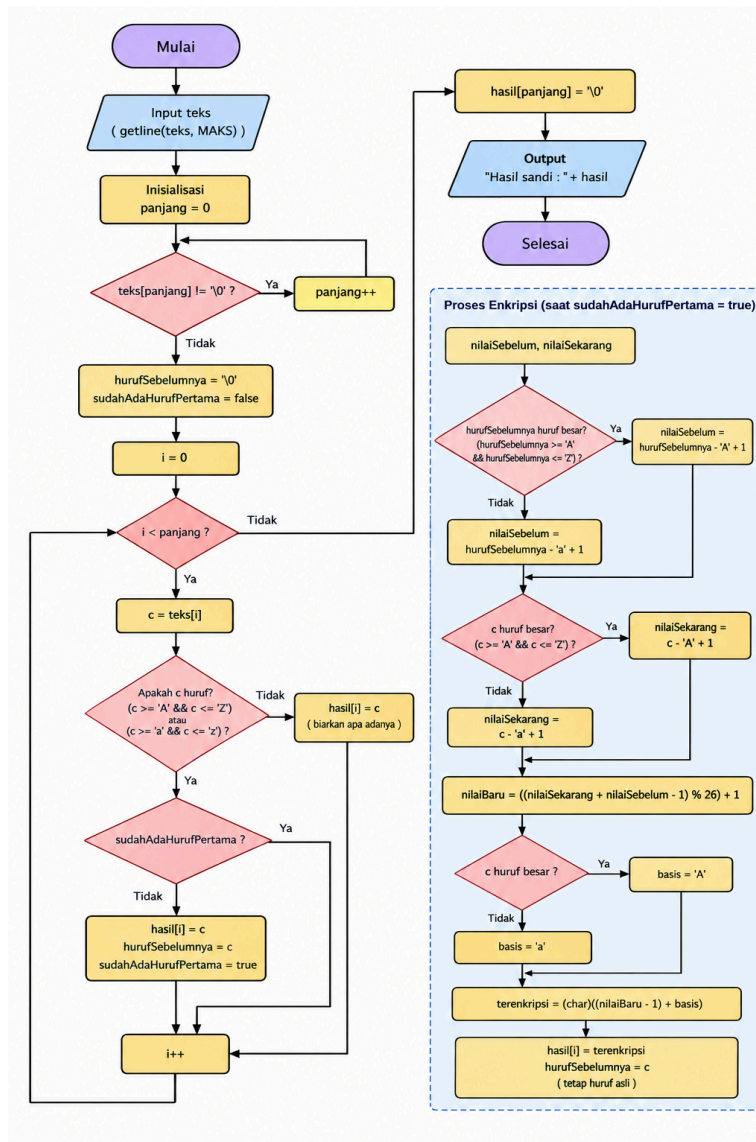
$\text{hurufSebelumnya} = c$  // acuan geser tetap huruf ASLI

$\text{hasil}[\text{panjang}] = '\0'$  // tutup string hasil secara manual

TAMPILKAN hasil

SELESAI

## 2. Flowchart



### 3. Penjelasan

BACA teks

HITUNG panjang teks secara manual (loop sampai ketemu '\0')

**Langkah 1** — Teks pesan dibaca sebagai larik karakter. Karena tidak boleh pakai fungsi library seperti `strlen()`, panjangnya dihitung manual dengan loop sampai ditemukan karakter penutup `'\0'`.

hurufSebelumnya = kosong  
sudahAdaHurufPertama = SALAH

**Langkah 2** — Dua variabel bantu disiapkan: `hurufSebelumnya` untuk menyimpan huruf asli terakhir yang diproses (dipakai sebagai acuan geser), dan `sudahAdaHurufPertama` sebagai penanda apakah huruf pertama sudah ditemukan (karena huruf pertama diperlakukan khusus, tidak berubah).

UNTUK  $i = 0$  SAMPAI  $\text{panjang} - 1$ :

```
c = teks[i]
hurufBesar = (c >= 'A' DAN c <= 'Z')
hurufKecil = (c >= 'a' DAN c <= 'z')
```

**Langkah 3** — Program memindai teks karakter per karakter. Untuk tiap karakter, dicek dulu apakah dia huruf besar atau huruf kecil — ini dilakukan manual lewat perbandingan kode ASCII, bukan pakai `isalpha()/isupper()`, sesuai batasan yang diminta.

JIKA (BUKAN hurufBesar) DAN (BUKAN hurufKecil) MAKA:

```
hasil[i] = c
LANJUT ke i berikutnya
```

**Langkah 4** — Kalau karakternya bukan huruf (misalnya spasi, tanda baca, angka), karakter itu langsung disalin apa adanya ke hasil, tanpa proses geser, dan lanjut ke karakter berikutnya.

JIKA sudahAdaHurufPertama = SALAH MAKA:

```
hasil[i] = c
hurufSebelumnya = c
sudahAdaHurufPertama = BENAR
```

**Langkah 5** — Kalau ini huruf **pertama** yang ditemukan dalam teks, huruf itu disalin tanpa perubahan (sesuai aturan soal), lalu disimpan sebagai `hurufSebelumnya` untuk dipakai sebagai acuan geser huruf berikutnya.

SELAIN ITU:

JIKA hurufSebelumnya huruf besar MAKA:

```
nilaiSebelum = hurufSebelumnya - 'A' + 1
```

SELAIN ITU:

```
nilaiSebelum = hurufSebelumnya - 'a' + 1
```

JIKA hurufBesar MAKA:

```
nilaiSekarang = c - 'A' + 1
```

SELAIN ITU:

```
nilaiSekarang = c - 'a' + 1
```

**Langkah 6** — Untuk huruf-huruf setelah huruf pertama, dihitung dulu nilai posisi alfabetnya ( $A/a=1 \dots Z/z=26$ ) — baik untuk huruf sebelumnya maupun huruf yang

sedang diproses sekarang. Perhitungan posisi ini pakai selisih kode ASCII terhadap 'A' atau 'a', tergantung apakah huruf itu besar atau kecil.

$$\text{nilaiBaru} = ((\text{nilaiSekarang} + \text{nilaiSebelum} - 1) \bmod 26) + 1$$

**Langkah 7** — Ini rumus inti pergeserannya: nilai huruf sekarang ditambah nilai huruf sebelumnya, lalu di-modulo 26 supaya kalau hasilnya melewati Z, otomatis "berputar" kembali ke A (wrap-around). Pengurangan/penambahan 1 di rumus ini menjaga penomoran tetap 1–26, bukan 0–25.

JIKA hurufBesar MAKA:

basis = 'A'

SELAIN ITU:

basis = 'a'

terenkripsi = (nilaiBaru - 1) + basis

hasil[i] = terenkripsi

hurufSebelumnya = c

**Langkah 8** — Nilai posisi hasil pergeseran (**nilaiBaru**) diubah kembali jadi karakter huruf, dengan mempertahankan case aslinya (besar/kecil). Huruf hasil sandi disimpan ke **hasil[i]**. Penting: **hurufSebelumnya** diperbarui dengan huruf **asli c**, bukan huruf hasil sandi — karena acuan geser untuk huruf berikutnya harus berdasarkan huruf asli, sesuai contoh di tabel sebelumnya.

hasil[panjang] = '\0'

TAMPILKAN hasil

SELESAI

**Langkah 9** — Setelah semua karakter selesai diproses, string hasil ditutup manual dengan karakter null (karena tidak pakai **std::string**), lalu ditampilkan sebagai pesan sandi akhir.